



STATION INTERVALS – GENERATIVE ART BETWEEN THE PROGRAMMES

DOC NO. RL-Z0-INTERVALS-A REV. A RIPOSTE LABORATORIES

Channel Z0 puts a piece of generative art at every junction in the schedule. The art is made by Glitchsheet – the same chain-of-typed-ops engine that makes the vinyl-cutter stickers – driven headlessly as a library rather than through its studio UI.

CONTENTS

▶ What is on screen	3
▶ How long each interval is	4
▶ The pipeline	5
1. Render – <code>tools/z0-generative.py</code> (LXC 114)	5
2. Assemble – <code>tools/z0-interval-assemble.py</code> (LXC 111)	6
3. Install (from x)	7
▶ The schedule	7
A rebuild needs a CONTENT change, not a touch	9
▶ Verifying it is actually on air	9
▶ Tuning	10

Roughly **40 minutes a day**, across **168 junctions** in the seven-day cycle.

A second family shares this pool: the **L-system specimens** in `lsystems.mdRL-Z0-LSYSTEMS`, where the program that drew the picture is on screen beside it. They carry the same `generative` tag, so they join the same shuffle and need no schedule change.

SEC.01

WHAT IS ON SCREEN

A **chain**: a numpy source generator, a palette, usually a form or colour treatment, and – about half the time – an **ffglitch codec pass**, which is real datamosh: the frames are encoded to MPEG-2, the motion vectors or quantisers are edited, and the damaged bit-stream is decoded back. That is the part that makes these look like Glitchsheet and not like a screensaver.

Each clip is one **piece**: the recipe (source, palette, stack) is fixed for its whole length while the seed and one parameter are **swept** across it, so a clip evolves rather than cutting between unrelated images. That sweep is Glitchsheet's own idea, borrowed directly from its variants panel.

Everything is seeded. The same `-seed` renders the same intervals, byte for byte, and every clip's exact chain is recorded in `manifest.json` next to it – so any interval on air can be traced back to the chain that made it and re-rendered at will.

SEC.02

HOW LONG EACH INTERVAL IS

The ask was that the first and last four minutes of every segment become generative art. Two properties of this schedule shaped how that is delivered.

Adjacent segments touch. Taken literally, the tail of one segment and the head of the next are eight unbroken minutes at every junction – about 1.9 hours a day. So each junction gets **one** interval, shared: still the last four minutes of the segment before and the first four of the segment after, counted once rather than twice.

Segments are wildly different lengths. CARTOON BLOCK runs two hours; THE WEATHER DESK is exactly two minutes and SIGN-OFF is thirty-five seconds. A flat four minutes would erase the short ones. So an interval is **12.5% of the shorter of the two segments it separates, capped at four minutes:**

INTERVAL	COUNT PER CYCLE	WHERE
4:00	35	between the long blocks – cartoons, Prelinger, the features
3:45 / 3:37	21	blocks a little under half an hour
1:52 / 1:15	37	either side of the shorter strands
0:15	35	either side of THE WEATHER DESK (2:00)
0:08 / 0:04	40	around SIGN-ON and SIGN-OFF (0:35)

168 intervals per seven-day cycle, **40 min a day.**

One pool of ~55-second clips serves all of them, because `duration:` + `trim: true` fills an exact wall-clock span and cuts the last item at the boundary.

The clock is preserved. Where the segment before a junction ends on a `pad_until`, that target is pulled BACK by the interval length, so the segment after still starts at the time the guide and the

storefront advertise – CARTOON BLOCK now pads to 08:56, the interval runs 08:56–09:00, and PRELINGER THEATRE still opens at 09:00. Where it ends on a fixed `count:` item (the weather, the sign-off, a feature) there is nothing to pull back and the following pad absorbs the shift. Intervals under a minute are never pulled back, because `pad_until` targets are only HH:MM.

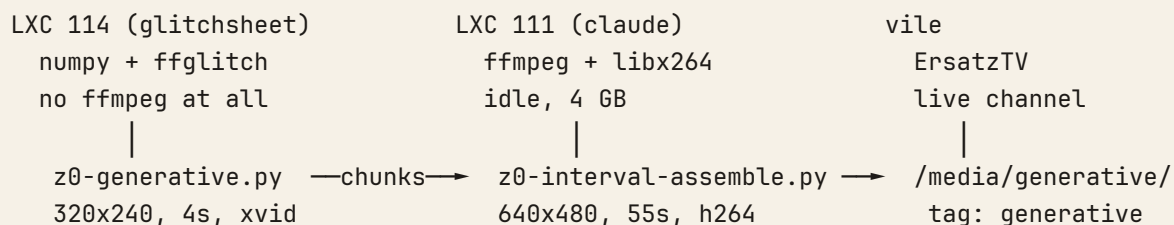
Graphics go down for the interval and come back after it. The bug and the weather card over abstract art look like a mistake – and the bottom strip is a subtitle element, the one that is re-rendered every frame and that took the channel off air 27 times in a morning. An hour or two a day where the compositor is idle is a saving, not a cost.

Intervals are **silent**, like the channel's own colour bars and test card.

SEC.03

THE PIPELINE

Three hosts, because no single one can do all of it.



1. RENDER – `tools/z0-generative.py` (LXC 114)

SH

```
pct exec 114 -- python3 /tmp/z0-generative.py \
  --clips 32 --chunks 18 --out /var/lib/z0gen/chunks
```

Chunks are **120 frames at 320x240**, and both numbers are forced:

- ▶ **320x240** doubles to the channel's 640x480 by an exact integer nearest-neighbour scale, so no interpolation softens the block edges the codec ops exist to create.
- ▶ **120 frames** is a memory ceiling, not the engine's 240-frame cap. Peak RSS is about 83 MB + 5.9 MB per frame at this size – 767 MB at 120 frames, 1137 MB at 180 – and the treatment stack lands on top of that in a 2 GB container that is also running the live studio. The heaviest stack renders at 120 and is OOM-killed at 150.

Each chunk renders in **its own subprocess**. numpy does not return freed arrays to the OS, so evaluating several chains in one interpreter stacks their peaks and the OOM killer takes the whole batch – exit 137, no traceback, no partial manifest. A subprocess per chunk turns an OOM into one lost chunk.

Roughly **20 s per chunk**; a 32-clip batch is about three hours. It resumes: clips already in `manifest.json` are skipped.

2. ASSEMBLE – `tools/z0-interval-assemble.py` (LXC 111)

SH

```
pct exec 111 -- python3 /tmp/z0-interval-assemble.py \
  --chunks-dir /root/z0gen/chunks --out-dir /root/z0gen/intervals
```

Crossfades the chunks (1 s, transition varied per clip), upscales 2x with `flags=neighbor`, adds a silent stereo 48 kHz AAC track and encodes H.264 at 3000k. Output is 4 + (N-1)x3 seconds – **55.0 s** for 18 chunks – and there is a length guard that refuses to install a file more than a second off, in the spirit of `tools/z0-weather.sh`.

Assembling on 111 rather than vile is deliberate: vile's live transcode runs at `-readrate 1.05`, a 5% margin, and x has 20 cores at load 0.5 against vile's 12 threads at load 2.2–3.0.

3. INSTALL (FROM x)

Clips go to `/media/generative/` on vile, which makes their tag `generative` (ErsatzTV tags Other Videos by folder). Install with `write-temp + mv -f`, never straight onto a live filename. Then let the app rescan – do NOT run the scanner yourself, it populates the DB but not the search index:

SH

```
sqlite3 /mnt/solid-state/ersatztv/ersatztv.sqlite3 \  
    "UPDATE LibraryPath SET LastScan = NULL WHERE Path = '/media';"  
# then restart the app
```

A brand-new tag needs the search index **REBUILT**, not just a scan. The pool is a `search:` key, so it is answered by Lucene rather than by the database. The first install here scanned cleanly – 30 rows, `Tag` populated, `LastScan` set – and the intervals still came out as zero items, because the index did not have the new `generative` tag. There is no error for this: an empty content pool is silently skipped, so it looks like the `duration:` instruction is unsupported.

SH

```
rm -rf /mnt/solid-state/ersatztv/search-index # then restart; it reindexes
```

Topping the pool up needs no schedule change. Drop more clips in the folder, rescan, and they join the shuffle.

SEC.04

THE SCHEDULE

Intervals are part of the **generated** week. `tools/z0-build-schedule.py` builds the seven days; `tools/z0_intervals.py` splices an interval into each day's finished instruction list, and the generator calls it unless you pass `--no-intervals`.

Splicing afterwards rather than emitting inline from `day_plan()` is deliberate:

an interval's length depends on **both** segments it sits between, which is only knowable once the day is complete.

SH

```
tools/z0-build-schedule.py --start-day thursday --out playout/channel-z0.yml
tools/validate-schedule.py playout/channel-z0.yml playout/sequential-schedule.schema.json
tools/validate-schedule.py playout/_content.yml playout/sequential-schedule-import.schema.json
tools/check-schedule-clock.py playout/channel-z0.yml
```

Build for **the day you are going to reset on** – the generator's `-start-day` handles the rotation, and a reset is mandatory after any change that shifts instruction indices, because `PlayoutAnchor` pins `NextInstructionIndex` and a Context `InstructionIndex` into the old list.

Validate the imported `_content.yml` separately: the main file no longer carries the content keys, so a schema check of it alone will not see them.

The last two checks are not ceremony:

- ▶ **validate-schedule.py** checks the file against ErsatzTV's own schema, pulled from `/app/Resources/sequential-schedule.schema.json` in the container. Every object is `additionalProperties: false`, and a failed validation makes `SequentialPlayoutBuilder` return `None` – the playout does not build **at all**, invisibly, while the items already built keep airing.
- ▶ **check-schedule-clock.py** walks the wall clock through the week and checks something the schema cannot: that every `pad_until` target is still in the future when the clock reaches it. A pad whose target has passed schedules **nothing** – a hole, not an error. It also reports the week's total pad span, which must stay at **167.5 h**; if it jumps by 24 h a target has rolled into the next day.

Also reset **early in the day**. A `pad_until` whose target is already past is skipped, so a mid-afternoon reset drops every morning block and the day starts wherever the clock has got to.

A REBUILD NEEDS A CONTENT CHANGE, NOT A TOUCH

ErsatzTV resets the playout by itself when the schedule file's **contents** change – `NextInstructionIndex` goes back to 0 and the week rebuilds on the next restart. touching the file is not enough; the mtime alone does not trigger it. Worth knowing, because the alternative is deleting `PlayoutItem`, `PlayoutAnchor` and `PlayoutHistory` by hand.

SEC.05

VERIFYING IT IS ACTUALLY ON AIR

The channel has a long history of every status surface looking healthy while something else is on screen, so check the picture, not the database.

SH

```
# 1. the pool is not empty – an empty pool is silently skipped, leaving a hole
sqlite3 /mnt/solid-state/ersatztv/ersatztv.sqlite3 \
  "SELECT COUNT(*) FROM Tag WHERE Name='generative';"

# 2. intervals actually reached the playout
sqlite3 /mnt/solid-state/ersatztv/ersatztv.sqlite3 \
  "SELECT COUNT(*) FROM PlayoutItem WHERE CustomTitle='STATION INTERVAL';"

# 3. look at the live edge – NOT the playlist, which starts at the OLDEST
#   segment and will happily hand you minutes-old colour bars
B=https://watch.ch0.ripostelabs.xyz
V=$(curl -s $B/hls/stream.m3u8 | grep -v '^#' | head -1)
SEG=$(curl -s $B/hls/$V | grep -v '^#' | tail -1)
ffmpeg -i "$B/hls/$(dirname $V)/$SEG" -frames:v 1 out.png
```

And after any ErsatzTV restart, **restart z0-uplink too** – it does not recover on its own, and Owncast will keep reporting `online: true` with a frozen picture.

SEC.06

TUNING

All in `tools/z0-generative.py`:

WANT	CHANGE
Calmer / busier	the probabilities in <code>build_recipe()</code>
Less / more damage	the <code>CODEC_SOFT</code> probability (0.45) and <code>CODEC_HARD</code> (0.12)
Gentler parameters overall	<code>lo_frac/hi_frac</code> in <code>random_params()</code> (currently the bottom half of every range)
Different palettes	<code>PALETTES</code> – the 10 in use are Glitchsheet's, minus as-is
Longer clips	<code>-chunks</code> (each adds 3 s), not <code>-frames</code>

`codec.storm` and `codec.bleed` are deliberately excluded. On a clean posterized base both produce structureless static at ~8 Mbps – no form survives them, and at the channel's 1200 kbps they arrive as grey mud. `codec.freeze` holds macroblocks instead of injecting noise and is the one heavy op kept.

Interval lengths are in `tools/z0_intervals.py`: `MAX_INTERVAL` (240 s), `MIN_INTERVAL` (4 s) and `FRACTION` (0.125 of the shorter neighbour). The `CONTENT_SECONDS` / `SEQUENCE_SECONDS` tables there are used **only** to size intervals – those numbers never reach the schedule, so being a minute out on a feature changes nothing.

SOURCE docs/intervals.tex SHA 3a51ea5c698e BUILT 2026-09-07